NET-NEUTRAL AI

Product Requirements Document

Decentralized Federated Learning Platform

GitHub DevDays Hackathon 2026 | Version 1.0 | May 2026

Document Owner	CS Team Member (Janhvesh)
Team	CS(Janhvesh) + IT(Tejas) + ENTC1(Bhoomika) + ENTC2(Atharv)
Status	Draft — v1.0
Last Updated	May 2026
Hackathon	GitHub DevDays 2026
Build Window	May 25 – June 10, 2026 (16 days)

1. Executive Summary

The One Paragraph That Matters

Today, training AI models costs millions of dollars and is accessible only to large tech companies. Billions of devices — student laptops, gaming PCs, smartphones — sit idle with capable GPUs doing nothing. Net-Neutral AI is a platform that recruits these idle devices as volunteer compute nodes, enabling collaborative AI model training without any single company controlling the infrastructure. Raw data never leaves a device; only learned weights are shared. Contributors earn credits for every training round they participate in. This is the net neutrality principle applied to AI compute: equal access, distributed power, no gatekeepers.

2. Problem Statement

2.1 The Centralization Problem

Modern large language model training requires compute resources that are prohibitively expensive for individuals, universities, and small organisations. This creates a structural monopoly: only a handful of companies can build frontier AI, concentrating both the capability and the societal influence of these systems.

Metric	Reality
Cost to train GPT-4 class model	~$100M+
Accessible to	< 5 organisations globally
Idle GPU capacity worldwide	Estimated billions of devices
Data privacy in centralised training	Raw data must leave user devices

2.2 The Opportunity

Federated Learning (FL) offers a proven paradigm: train models collaboratively across distributed devices without centralising raw data. However, existing FL systems are either research prototypes or proprietary corporate tools. No open, incentivised, volunteer-driven FL platform exists at a student-accessible level.

Core Insight

If you can coordinate idle GPUs the way BitTorrent coordinates idle bandwidth, you can democratise AI training. Net-Neutral AI is that coordination layer.

3. Goals & Non-Goals

3.1 Goals — Hackathon Prototype

- Demonstrate federated learning across 3 physical laptops on a shared WiFi network
- Train a small Transformer model (DistilBERT) collaboratively using FedAvg
- Ensure raw training data never leaves any client device
- Implement a persistent credit ledger that tracks and rewards each node's compute contribution
- Show measurable model accuracy improvement across training rounds
- Present a live, crashing-free demo to hackathon judges

3.2 Non-Goals — Explicitly Out of Scope

- Internet-based communication across different networks (future milestone)
- Blockchain-based credit system — SQLite ledger is sufficient for prototype
- Training models larger than DistilBERT (~66M parameters)
- Mobile device participation
- Homomorphic encryption or advanced secure aggregation
- Real user onboarding or production deployment

4. Users & Stakeholders

User / Stakeholder	Role	Primary Need
Hackathon Judges	Primary evaluators of the prototype	See a working demo, understand the vision, assess technical depth
CS Team Member	Builds ML pipeline + FedAvg	Clear spec of what the model must do each round
IT Team Member	Builds coordinator server + credit DB	Clear API contract between server and clients
ENTC Team Members	Build client app + network layer	Clear spec of what the client sends and receives
Future Volunteer Nodes	Real-world end users (post-hackathon)	Easy installation, transparent credit tracking, privacy guarantee

5. Product Requirements — Judge-Facing View

This section describes what the product must demonstrate to judges. These are the criteria that win the hackathon.

5.1 Scalability & Need (Core Pitch)

Why This Matters to Judges

The judging criteria explicitly includes Innovation & Creativity and Practical Impact. The scalability argument is your strongest card: this architecture scales from 3 laptops to 3 million devices with zero architectural changes. Make judges feel that.

- The system must visibly work with 3 nodes and the architecture diagram must show how it extends to N nodes
- The pitch must articulate the real-world need: AI training cost monopoly, idle compute waste, data privacy violation in centralised training
- The credit system must suggest a real economic incentive model even if simplified for demo

5.2 Must-Have Demo Moments

Demo Beat	What Judges See	Why It Lands
Node Registration	3 terminals connect to coordinator, names appear on dashboard	Proves the network is real, not simulated
Local Training	Each client trains on its own data slice, loss prints to terminal	Shows data privacy: data stays local
Weight Upload	Clients POST weight updates to coordinator	Core FL mechanic made visible
FedAvg Round	Coordinator averages weights, round counter increments	The intelligence aggregation moment
Accuracy Improvement	Global model accuracy improves round over round	Proof it actually works
Credit Leaderboard	Dashboard shows each node's earned credits ticking up	The incentive model made tangible

5.3 GitHub & AI Tool Requirements

This hackathon scores on effective use of GitHub and AI tools. The following are required:

- Active public GitHub repository with meaningful commit history throughout build phase
- README that explains setup, architecture, and how to run the demo
- Use of GitHub Copilot or similar AI coding assistant — document where it was used
- At least one GitHub Actions workflow (even a simple lint/test pipeline counts)

6. Technical Requirements — Developer View

This section is for the build team. It specifies exactly what each component must do, accept, and return.

6.1 Network Constraints

- All 3 laptops must be on the same WiFi network during demo
- The coordinator's IP address must be known to all clients before the demo (hardcoded or config file)
- No internet dependency during the demo — the entire system runs on local network
- Target round-trip latency: under 5 seconds per training round on local WiFi

6.2 Coordinator Server (IT)

Endpoint	Method	Input	Output
/register	POST	{ client_id: string }	{ status: 'ok' }
/model	GET	None	Current global model weights (state_dict as JSON/binary)
/submit	POST	{ client_id, weights, samples_trained, time_seconds }	{ credits_earned, round }
/status	GET	None	{ round, clients, leaderboard }

- Framework: Flask (Python)
- Database: SQLite — one table for credits (client_id, round, samples, time_seconds, points)
- FedAvg logic: receives N weight dicts, returns element-wise mean across all layers
- Must handle a client dropping mid-round — skip missing client and continue aggregation
- Credit formula: points = floor(samples_trained / 5)

6.3 Client Application (ENTC)

- Language: Python script, command-line invocation
- On start: registers with coordinator, downloads current global model
- Training loop: loads local data shard, runs 1-3 epochs, computes updated weights
- On finish: POSTs updated weights + metadata to /submit endpoint
- Repeats for N rounds (configurable, default 5 rounds for demo)
- Must print round number, local loss, and credits earned after each round

6.4 ML Pipeline (CS — Janhvesh)

Component	Specification

Base Model	DistilBERT (distilbert-base-uncased) via HuggingFace Transformers
Task	Binary text classification (sentiment) on IMDb dataset
Dataset Split	50K reviews split evenly: ~16.6K per client (IID split for demo)
Local Training	1–3 epochs per round, AdamW optimizer, lr=2e-5
Weight Format	PyTorch state_dict serialised to binary via torch.save / io.BytesIO
FedAvg	Element-wise mean of all client state_dicts, applied to global model
Evaluation	Global model evaluated on shared 2K validation set after each round
Metric	Accuracy (%) — must show upward trend across rounds for demo

6.5 Credit Ledger Schema

Column	Type	Description
id	INTEGER PRIMARY KEY	Auto-increment row ID
client_id	TEXT NOT NULL	Unique identifier for each node (e.g. 'client_A')
round	INTEGER NOT NULL	Training round number
samples_trained	INTEGER	Number of data samples processed this round
time_seconds	REAL	Wall-clock seconds for local training
points_earned	INTEGER	Computed as floor(samples_trained / 5)
timestamp	DATETIME	Server timestamp of submission

7. Success Metrics

Metric	Target	Kill Switch (Pivot if...)
Demo completes without crash	100% of rounds complete	Falls back to screen recording of working run
Model accuracy improvement	At least +5% from round 1 to round 5	Reduce to 3 rounds, increase epochs per round
3 nodes active simultaneously	All 3 laptops online during demo	Simulate 3 clients on 1 machine as fallback
Credit leaderboard visible	Live update after each round	Static table in slides as fallback
GitHub repo activity	Commits from all team members	Minimum: daily commits from CS + IT

8. Risks & Mitigations

Risk	Likelihood	Mitigation
WiFi instability during demo	Medium	Test on same network day before. Have screen recording as backup.
DistilBERT too slow to train live	High	Pre-train to round 3, demo rounds 4-5 live. Or use TinyBERT.
Team member doesn't deliver their module	Medium	CS + IT are the critical path. ENTC can be simulated on localhost if needed.
FedAvg not converging in 5 rounds	Low	Use pre-split IID data. Convergence is almost guaranteed with balanced data.
Judges ask about blockchain / encryption	High	Prepare answer: 'Out of scope for prototype, here is the architectural plan for v2.'

9. Future Roadmap (Post-Hackathon)

This section shows judges the product has legs beyond the demo. It also maps to the resume value of continuing to build this.

Phase	Timeline	What Gets Added
v1 — Prototype	Now (Hackathon)	3-node local WiFi demo, FedAvg, SQLite credits, DistilBERT
v2 — Internet-Ready	Post-hackathon (1–2 months)	Cloud coordinator, internet node communication, secure HTTPS
v3 — Privacy Layer	3–4 months	Differential privacy on gradients, secure aggregation
v4 — Scale Test	6 months	10+ real volunteer nodes, larger model, async training rounds
v5 — Open Platform	1 year	Public client installer, real credit system, community governance

10. Open Questions

- Will DistilBERT fine-tuning complete in under 60 seconds per round on consumer laptops? (needs profiling in week 1)
- Should the coordinator run on one of the 3 demo laptops or on a 4th dedicated machine?
- How will the dashboard be displayed during demo — browser, terminal, or projector screen?

- Does the hackathon require Microsoft tools integration? If yes, Azure hosting of coordinator is a quick win.

Appendix: Glossary

Term	Plain English Definition
Federated Learning (FL)	Training an AI model across multiple devices without sharing raw data
FedAvg	Federated Averaging — the algorithm that combines weight updates from all clients into one global model
state_dict	PyTorch's way of storing a model's learned weights as a dictionary of tensors
IID Split	Independent and Identically Distributed — each client gets a balanced, representative data slice
Non-IID	Skewed data distribution across clients — what happens in the real world, not in our demo
Credit Ledger	A database table tracking how much compute each node contributed, and their reward points
Coordinator	The central Flask server that distributes the model, runs FedAvg, and logs credits
Client Node	A laptop running the client script, training locally and sending weight updates to coordinator